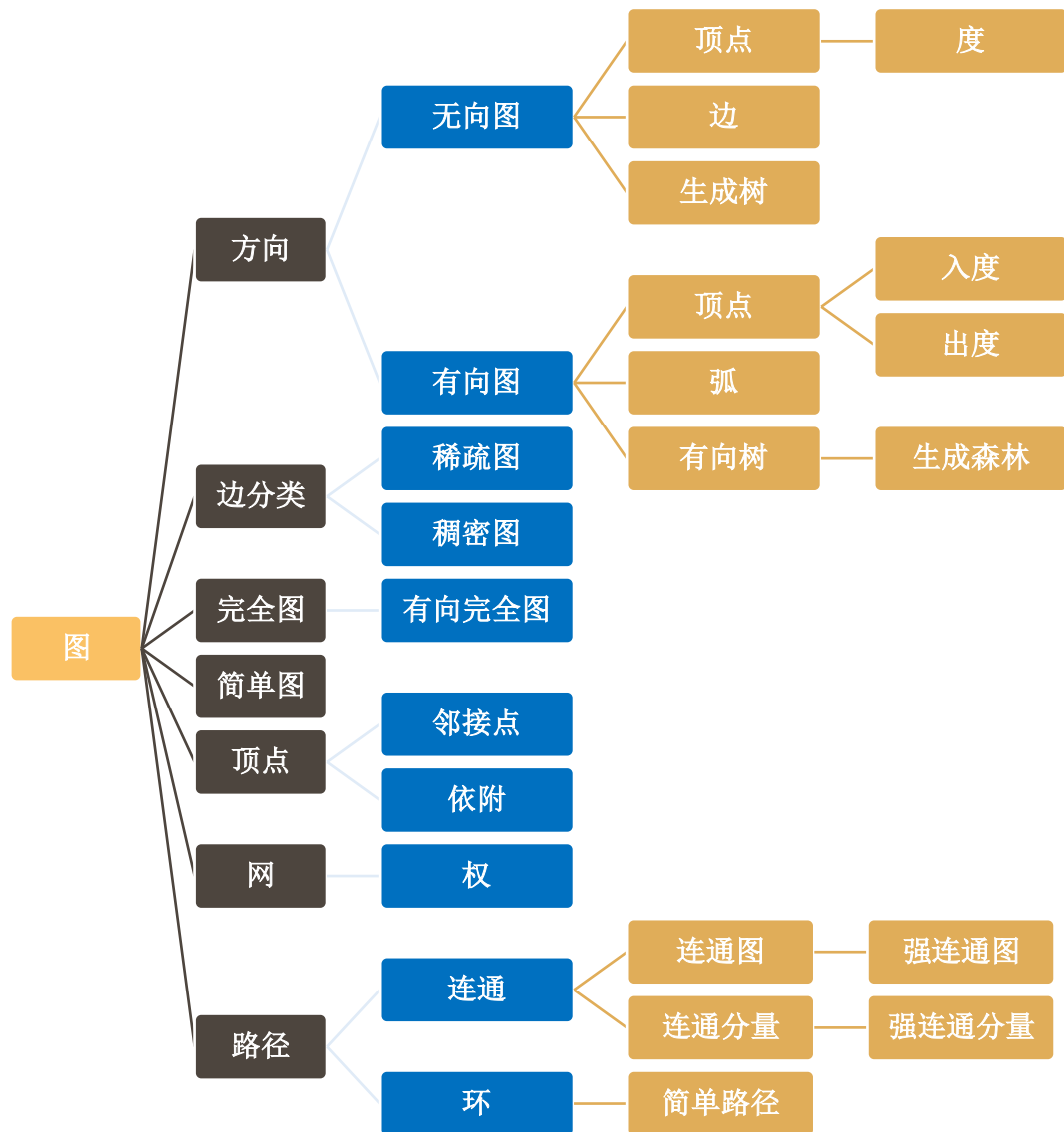


第7章 图



回顾



回顾

邻接矩阵是表示顶点之间相邻关系的矩阵。设 $G=(V,VR)$ 是具有 $n(n>0)$ 个顶点的图，顶点的顺序依次为 $(v_0, v_1, \dots, v_{n-1})$ ，则 G 的邻接矩阵 A 是 n 阶方阵，其定义如下：

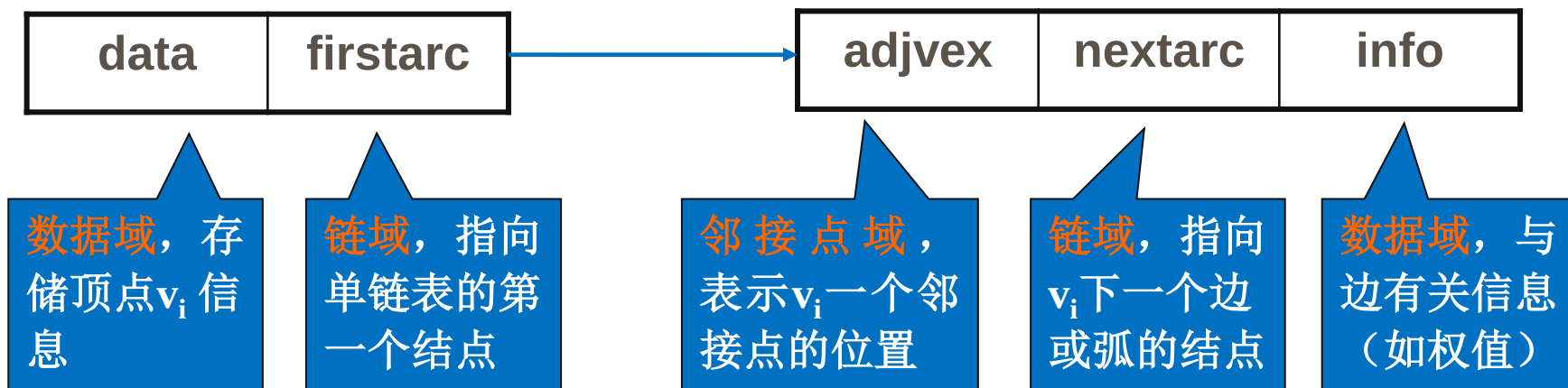
- ◆ 如果 G 是**无向图**，则： $A[i][j] = 1$:若 $(v_i, v_j) \in VR$
 0 :其他
- ◆ 如果 G 是**有向图**，则： $A[i][j] = 1$: $\langle v_i, v_j \rangle \in VR$
 0 :其他
- ◆ 如果 G 是**带权无向图**，则： $A[i][j] = w_{ij}$:若 $v_i \neq v_j$ 且 $(v_i, v_j) \in VR$
 ∞ :其他
- ◆ 如果 G 是**带权有向图**，则： $A[i][j] = w_{ij}$:若 $v_i \neq v_j$ 且 $\langle v_i, v_j \rangle \in VR$
 ∞ :其他

回顾

邻接表：对每个顶点 v_i 建立一个单链表，把与 v_i 有关联的边的信息链接起来，每个结点设为3个域；

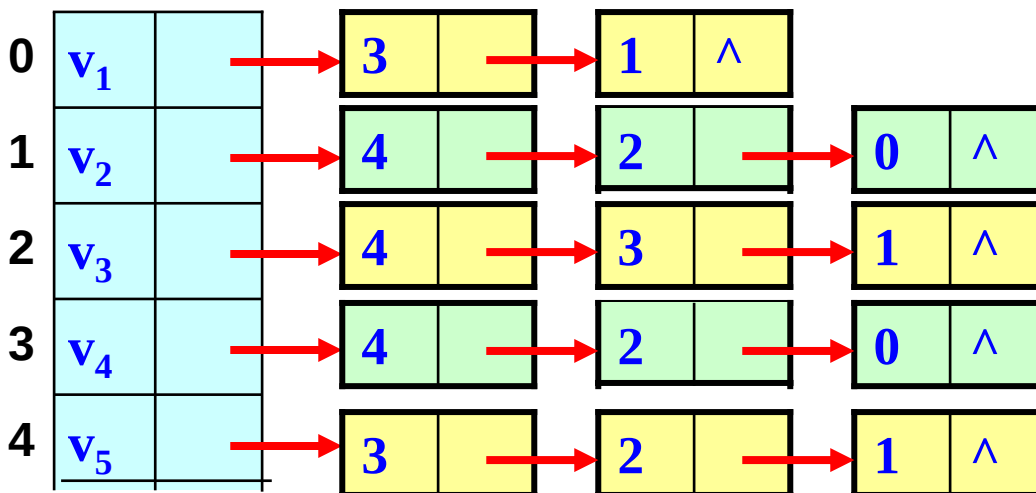
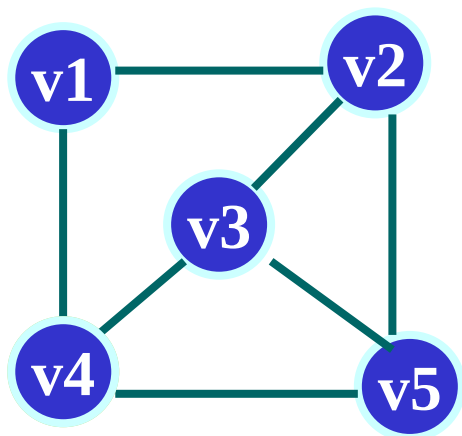
头结点

表结点



- 每个单链表有一个头结点（设为2个域），存 v_i 信息；
- 每个单链表的头结点通常以顺序存储结构存储。

回顾



(v_1 v_2 v_3 v_4 v_5)

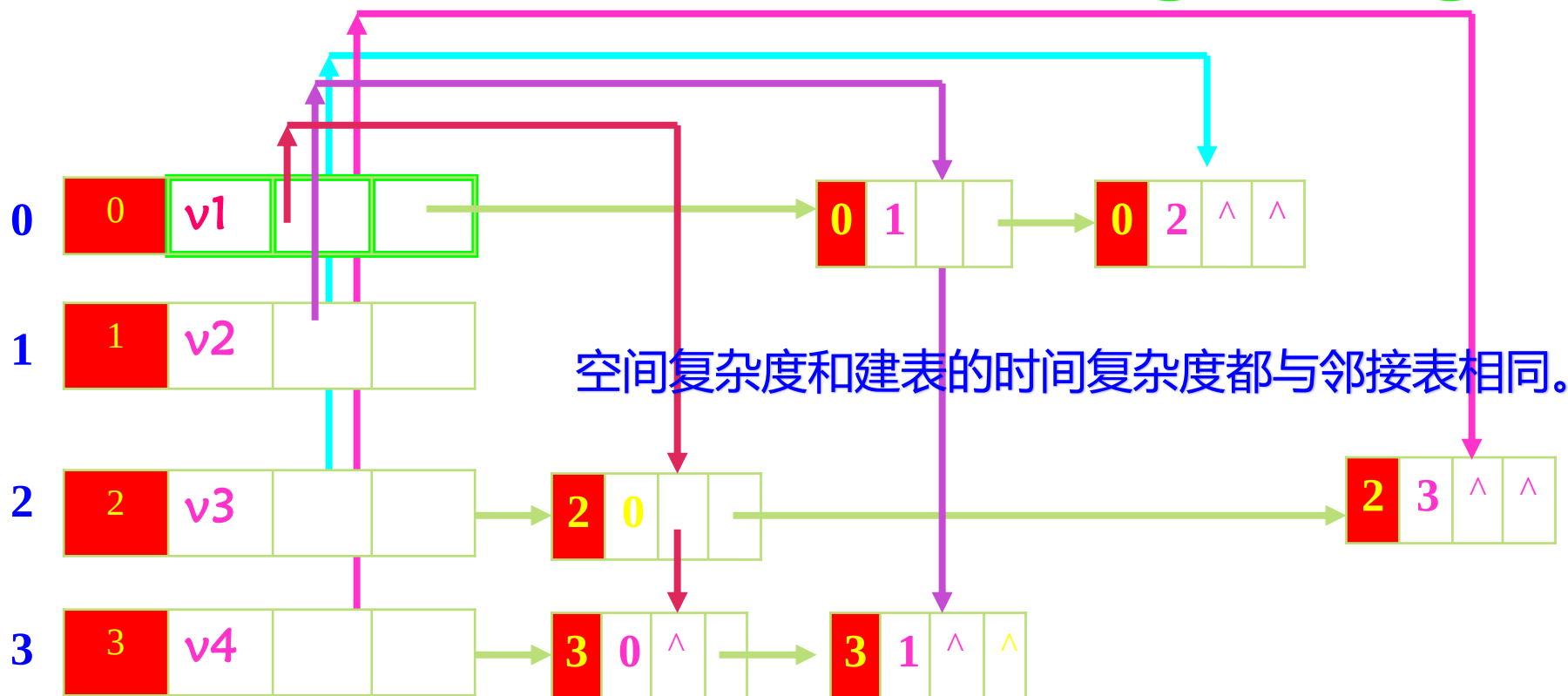
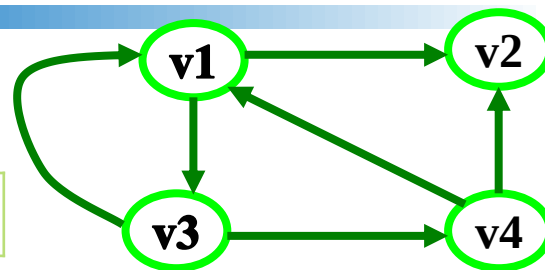
$$\begin{matrix} v_1 \\ v_2 \\ v_3 \\ v_4 \\ v_5 \end{matrix} \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 \end{bmatrix}$$

- **联系：**邻接表中每个链表对应于邻接矩阵中的一行，链表中结点个数等于一行中非零元素的个数。

回顾

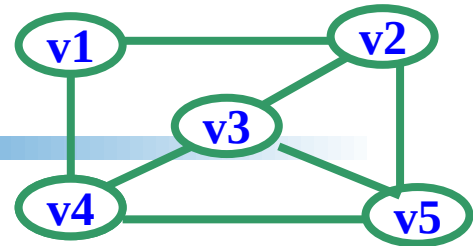
顶点结点	data	Firstin	Firstout
------	------	---------	----------

弧结点	tailvex	headvex	hlink	tlink	info
-----	---------	---------	-------	-------	------



十字链表：容易操作，如求顶点的入度、出度等。

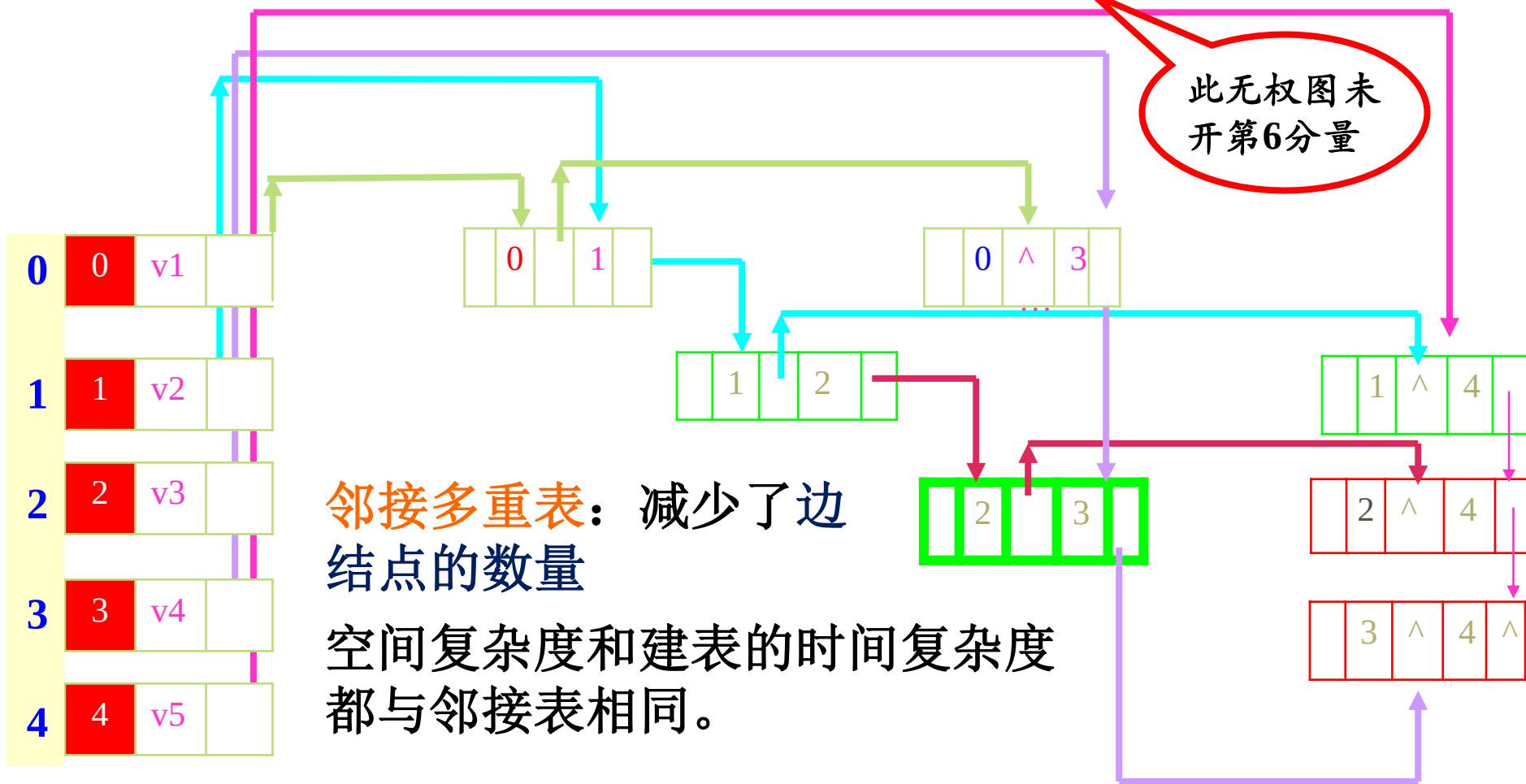
回顾



顶点结点

边结点

data	Firstedge	mark	ivex	ilink	jvex	jlink	info
------	-----------	------	------	-------	------	-------	------



邻接多重表：减少了边结点的数量

空间复杂度和建表的时间复杂度都与邻接表相同。

第7章 图

7.1 图的定义和术语

7.2 图的存储结构

7.3 图的遍历

7.4 最小生成树

7.5 活动网络

7.6 最短路径

7.3 图的遍历

旅行商问题(TSP, traveling salesman problem)

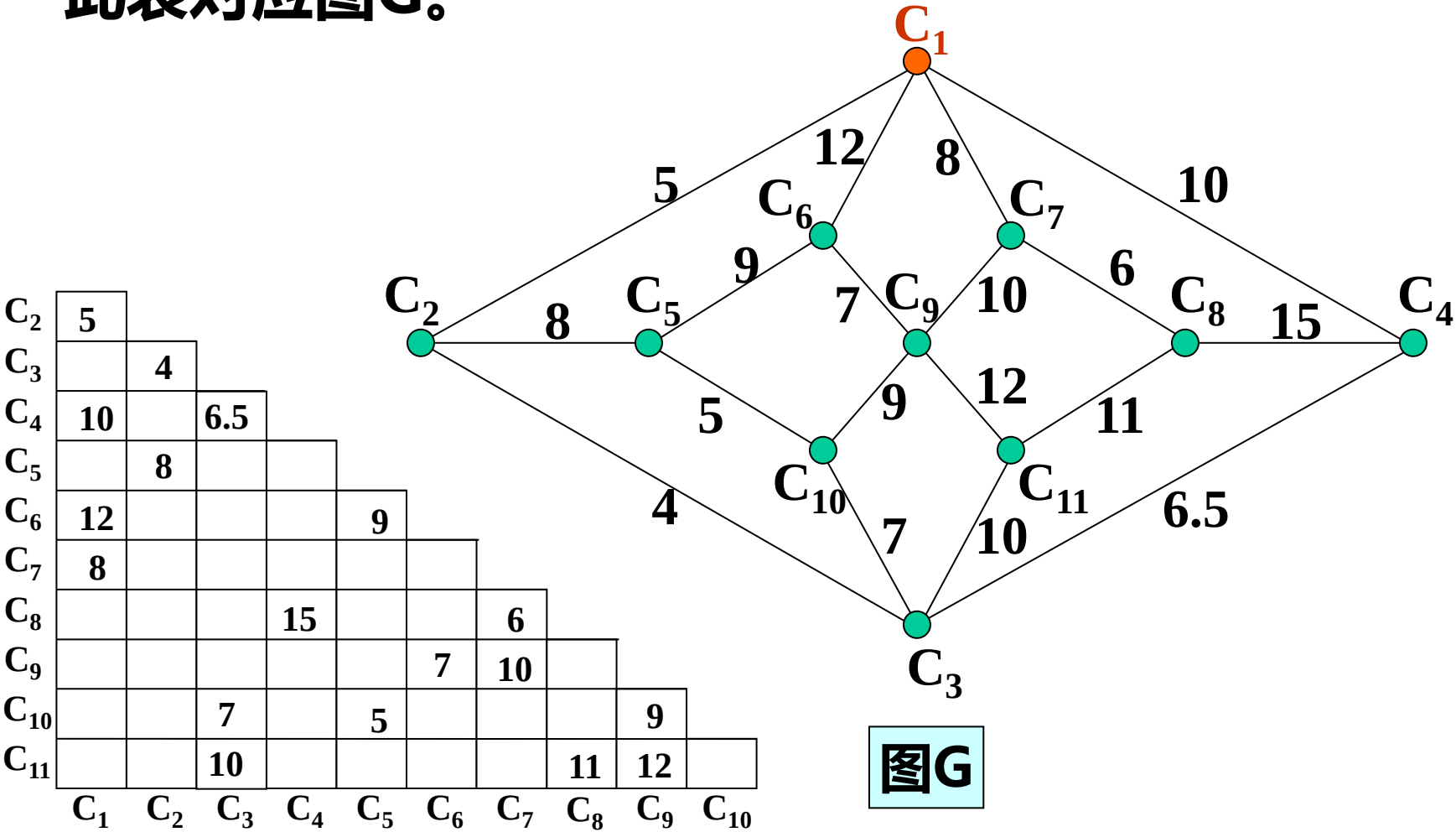
一个商人欲到 n 个城市推销商品，每两个城市 i 和 j 之间的距离为 d_{ij} ，如何选择一条道路使得商人每个城市正好走一遍后回到起点且所走路线最短。

C_2	5									
C_3		4								
C_4	10		6.5							
C_5		8								
C_6	12				9					
C_7	8									
C_8				15			6			
C_9						7	10			
C_{10}			7		5				9	
C_{11}			10					11	12	
	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8	C_9	C_{10}

单位：十公里

7.3 图的遍历

此表对应图G。



7.3 图的遍历

- **图的遍历**: 从图中某一顶点出发遍历图中其余顶点, 使得每个结点均被访问一次, 而且仅被访问一次。
 - **遍历实质**: 找每个顶点的邻接点的过程。
 - **遍历方法**: 深度优先搜索DFS (Depth_First Search)
广度优先搜索BFS(Breadth_First Search)
- **图的特点**: 图中可能存在**回路**, 且图的任一顶点都可能与其它顶点相通, 在访问完某个顶点之后可能会沿着某些边又**回到了曾经访问过的顶点**。

7.3 图的遍历

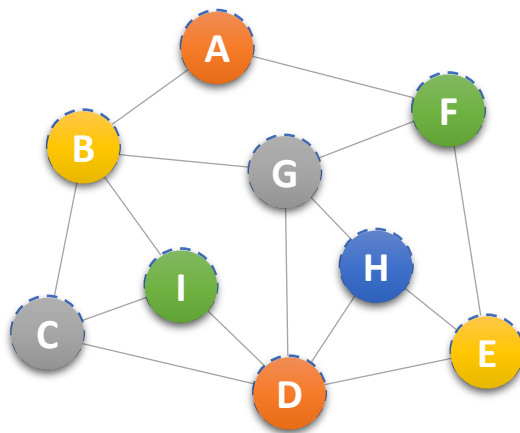
怎样避免重复访问?

解决思路：可设置一个辅助数组 *visited* [*n*]，用来标记每个被访问过的顶点。它的初始状态为0，在图的遍历过程中，一旦某一个顶点 *i* 被访问，就立即改 *visited* [*i*] 为1，防止它被多次访问。

7.3.1 深度优先遍历

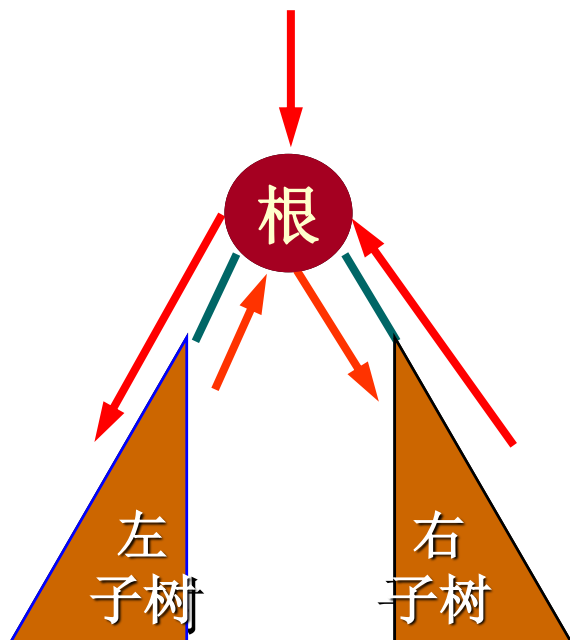
□ 连通图的深度优先搜索遍历

从图中某个顶点**V0** 出发，访问此顶点，然后依次从**V0**的各个未被访问的邻接点出发**深度优先搜索遍历**图，直至图中所有和**V0**有路径相通的顶点都被访问到。



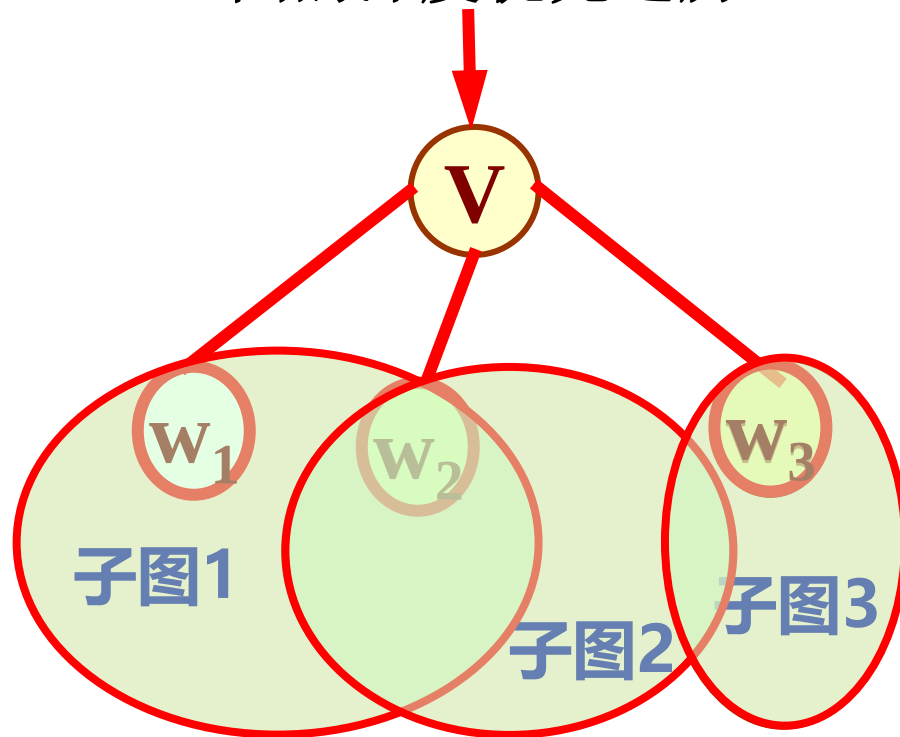
7.3.1 深度优先遍历

二叉树的先根遍历



1. 访问根结点；
2. 先序遍历左子树；
3. 先序遍历右子树。

图的深度优先遍历

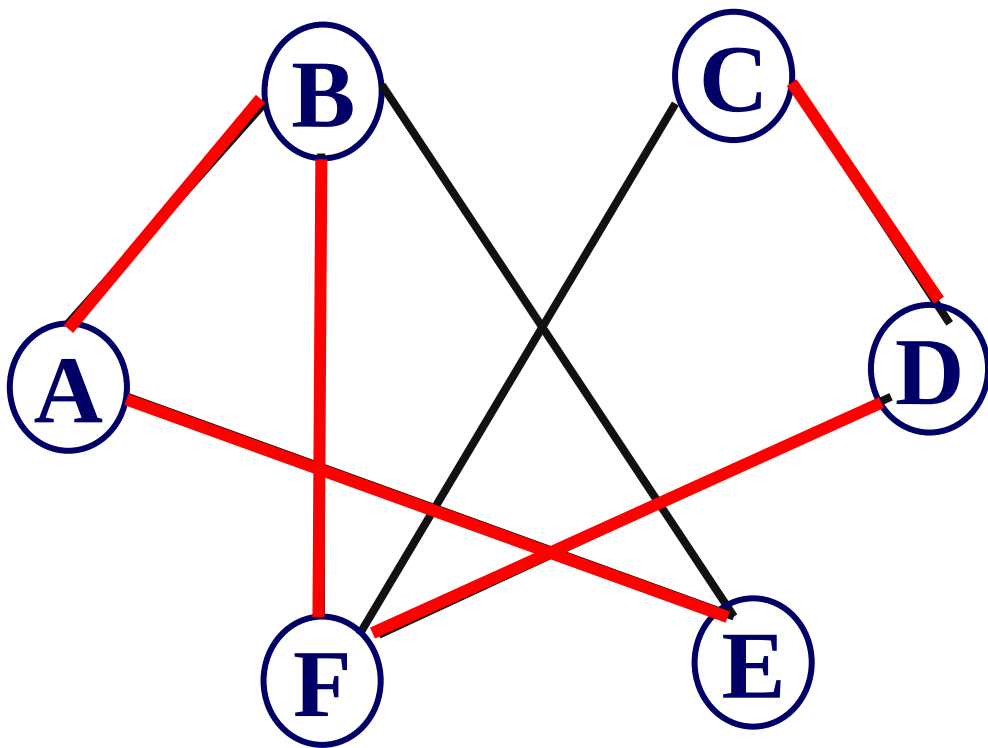


1. 访问顶点 V ；
2. for (W_1 、 W_2 、 W_3)
若该邻接点 W_i 未被访问，
则深度优先搜索遍历该子图。

7.3.1 深度优先遍历

深度优先搜索举例：

从B点开始遍历 I. **BAEFDC**



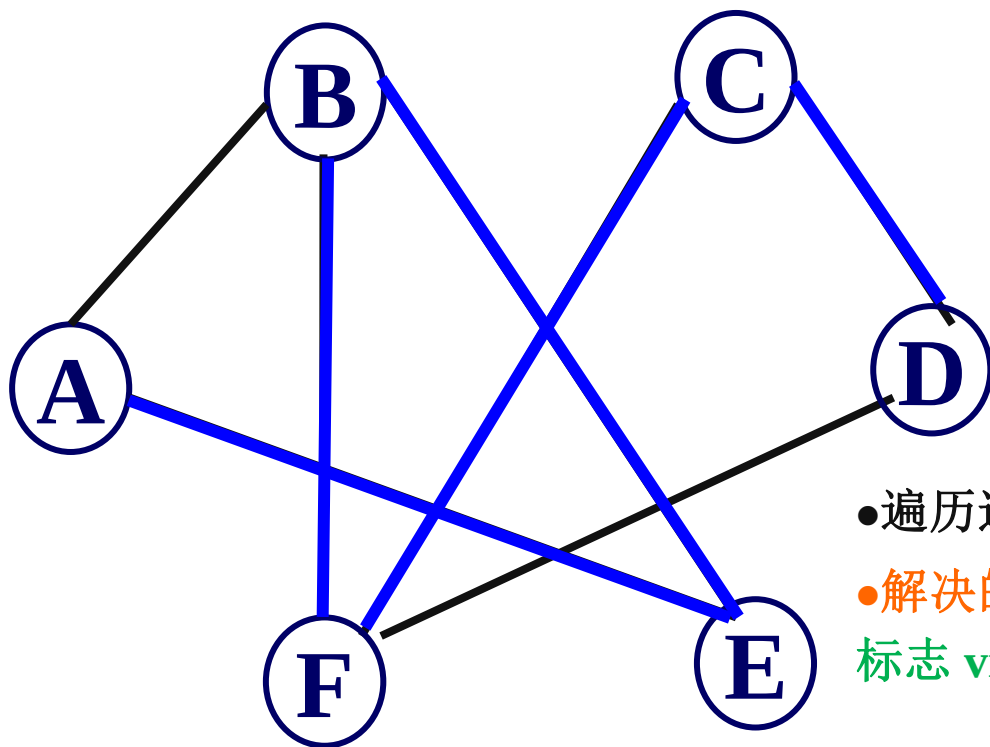
7.3.1 深度优先遍历

深度优先搜索举例：

从B点开始遍历

I. BAEFDC

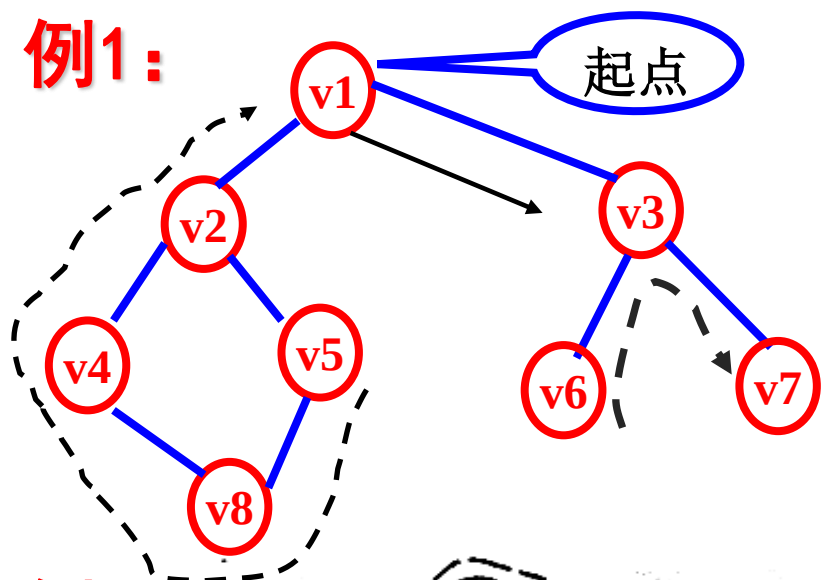
II. BEAFCD



- 遍历过程需要判别V的邻接点是否被访问？
- 解决的办法：为每个顶点设立一个“访问标志 `visited[w]`”；

7.3.1 深度优先遍历

例1:

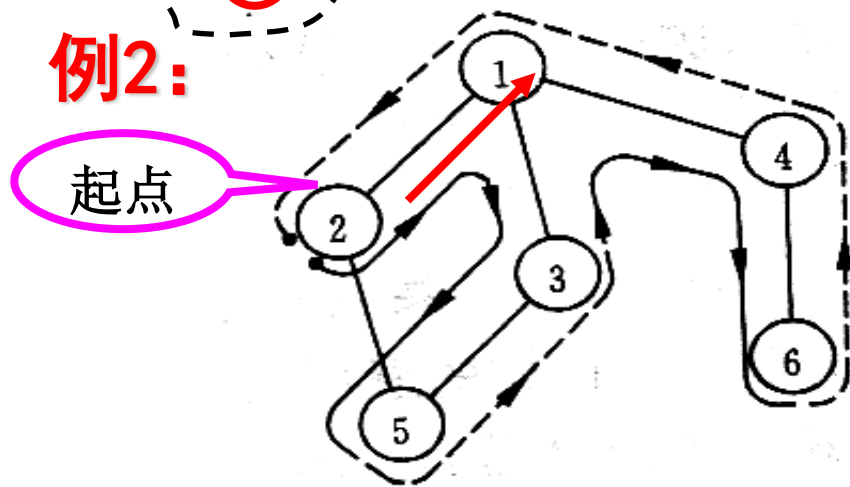


DFS 结果

$v1 \rightarrow v2 \rightarrow v4 \rightarrow v8 \rightarrow v5 \rightarrow v3 \rightarrow v6 \rightarrow v7$

应退回到v8, 因为v2已有标记

例2:



DFS 结果

$v2 \rightarrow v1 \rightarrow v3 \rightarrow v5 \rightarrow v4 \rightarrow v6$

7.3.1 深度优先遍历

简单归纳：

1. 访问起始点 v ;
2. 若 v 的第1个邻接点没访问过，深度遍历此邻接点;
3. 若当前邻接点已访问过，再找 v 的第2个邻接点重新遍历。

7.3.1 深度优先遍历

讨论1: 计算机如何实现DFS? ——开辅助数组 *visited* [n]!

邻接矩阵 A

	1	2	3	4	5	6
1	0	1	1	1	0	0
2	1	0	0	0	1	0
3	1	0	0	0	1	0
4	1	0	0	0	0	1
5	0	1	1	0	0	0
6	0	0	0	1	0	0

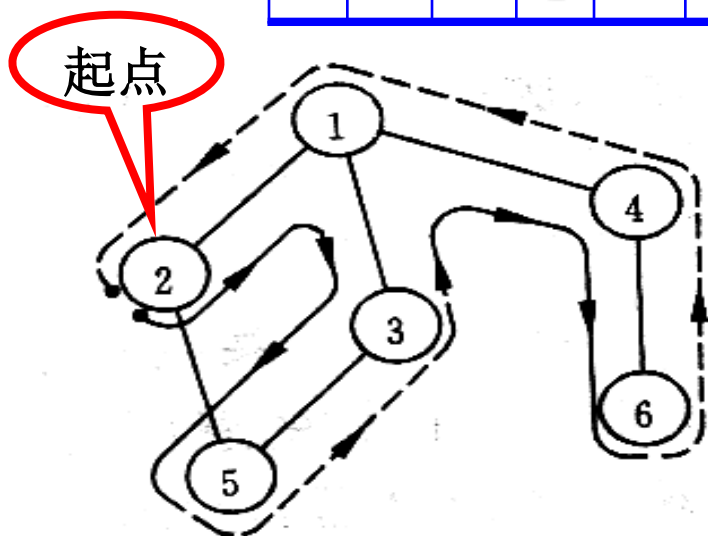
辅助数组 *visited* [n]

	1	2	3	4	5	6
1	0	0	1	1	1	1
2	0	1	1	1	1	1
3	0	0	0	1	1	1
4	0	0	0	0	0	1
5	0	0	0	0	1	1
6	0	0	0	0	0	1

DFS 结果

$v_2 \rightarrow v_1 \rightarrow v_3 \rightarrow v_5 \rightarrow v_4 \rightarrow v_6$

请注意逐级回退是递归概念



7.3.1 深度优先遍历

讨论2: DFS算法如何编程?

——可以用递归算法!

```
DFS (A, n, v) {  
    visit (v) ;      //访问 (例如打印) 顶点v  
    visited[v]=1;    //访问后立即修改辅助数组标志  
    for( j=1; j<=n; j++) //从v所在行从头搜索邻接点  
        if ( A[v, j] && ! visited[j] ) DFS (A, n, j);  
    return;  
} // DFS
```

$A[v,j] = 1$
有邻接点

$visited[j] = 0$
未访问过

7.3.1 深度优先遍历

```
void DFS(Graph G, int v) {
```

```
    // 从顶点v出发，深度优先搜索遍历连通图 G
```

```
    visited[v] = TRUE; VisitFunc(v); //访问第V个顶点
```

```
    for(w=FirstAdjVex(G, v); w >= 0 ; w=NextAdjVex(G,v,w))
```

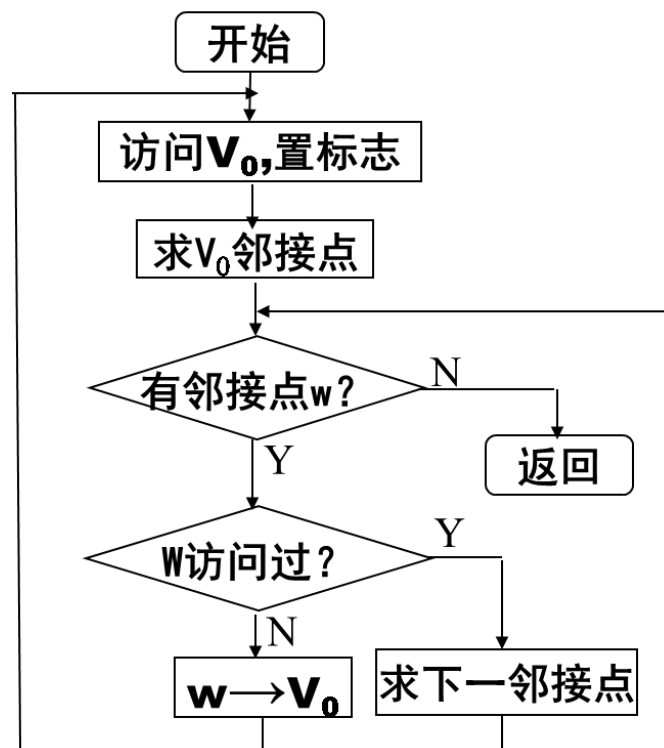
```
    { if (!visited[w]) DFS(G, w);
```

```
        // 对v的尚未访问的邻接顶点w
```

```
        // 递归调用DFS
```

```
    }
```

```
} // DFS
```



7.3.1 深度优先遍历

□ 基于邻接表的深度优先搜索举例

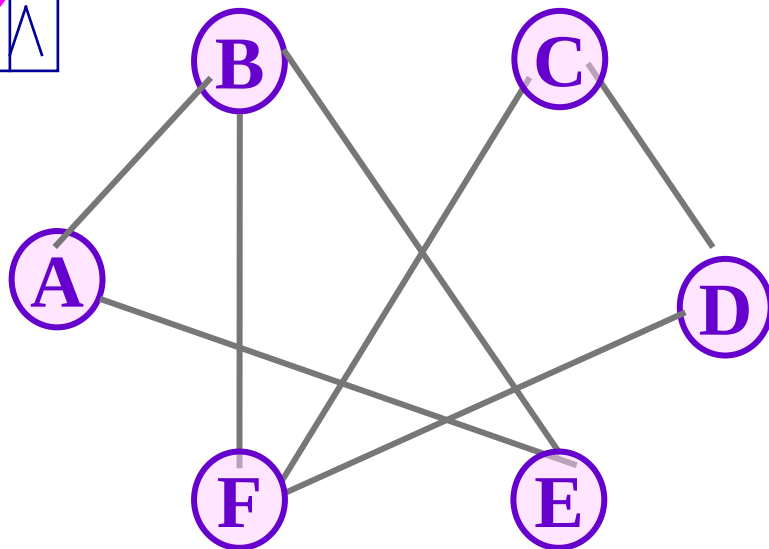
A, B, C, D, E, F

visited { **0**, **0**, **0**, **0**, **0**, **0** }

从A点开始遍历

A B E F C D

0	A	→	1	→	4 ^
1	B	→	0	→	4 → 5 ^
2	C	→	3	→	5 ^
3	D	→	2	→	5 ^
4	E	→	0	→	1 ^
5	F	→	1	→	2 → 3 ^



7.3.1 深度优先遍历

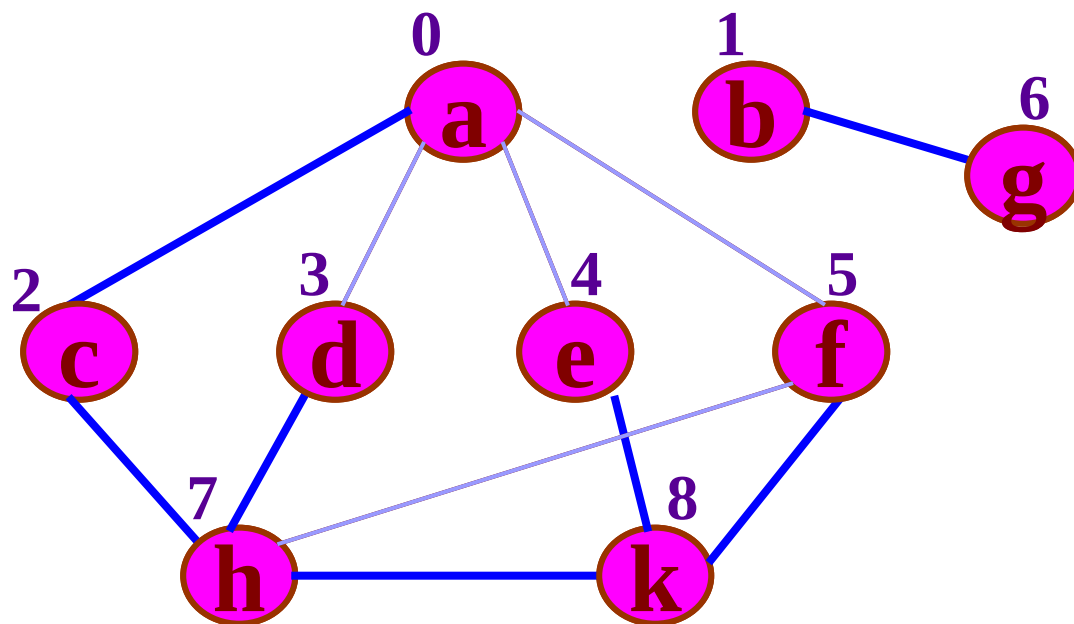
```
void DFS_AL(ALGraph G, int i, int n) {  
    // 从顶点Vi出发，深度优先搜索遍历邻接表表示的图G  
    visited[i] = TRUE;           //标记Vi已被访问过  
    VisitFunc(i);                //输出Vi  
    p= G.vertices[i].firstarc;  
    while(p){                    //依次搜索Vi的每个邻接点  
        j=p->adjvex;  
        if (! visited[j] )      //若Vj未被访问过，则递归调用  
            DFS _AL(G, j, n);  
        p=p-> nextarc;          //使p指向vi单链表的下一个邻接点  
    }//while  
} // DFS _AL
```

7.3.1 深度优先遍历

□ 非连通图的深度优先搜索遍历

首先将图中每个顶点的访问标志设为 `FALSE`，之后搜索图中每个顶点，如果未被访问，则以该顶点为起始点，进行深度优先搜索遍历，否则继续检查下一顶点。

7.3.1 深度优先遍历



访问标志:

0	1	2	3	4	5	6	7	8
T	T	T	T	T	T	T	T	T

访问次序:

a	c	h	d	k	f	e	b	g
---	---	---	---	---	---	---	---	---

7.3.1 深度优先遍历

深度优先搜索遍历非连通图 G 的算法描述

```
void DFSTraverse(Graph G, Status (*Visit)(int v))
```

```
{// 对图 G 作深度优先遍历。
```

```
    VisitFunc = Visit; //初始化访问函数
```

```
    for (v=0; v<G.vexnum; ++v)
```

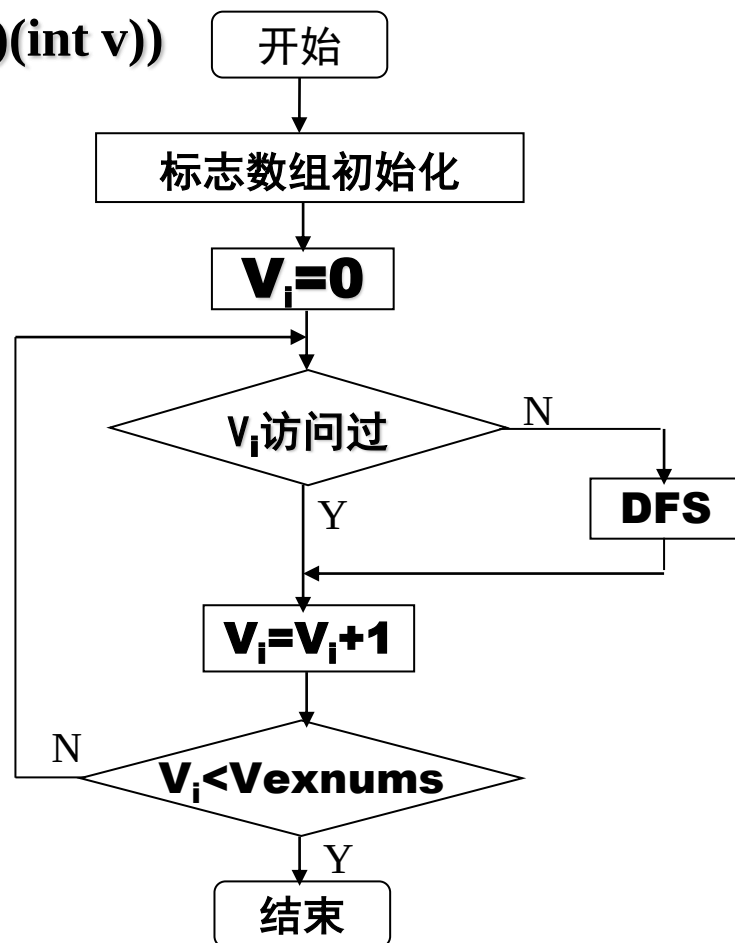
```
        visited[v] = FALSE; // 访问标志数组初始化
```

```
    for (v=0; v<G.vexnum; ++v)
```

```
        if (!visited[v]) DFS(G, v);
```

```
        // 对尚未访问的顶点调用DFS
```

```
}
```



7.3.1 深度优先遍历

DFS 算法效率分析（设图中有 n 个顶点， e 条边）

- 如果用邻接矩阵来表示图，遍历图中每一个顶点都要从头扫描该顶点所在行，因此遍历全部顶点所需的时间为 $O(n^2)$ 。
- 如果用邻接表来表示图，虽然有 $2e$ 个表结点，但只需扫描 e 个结点即可完成遍历，加上访问 n 个头结点的时间，因此遍历图的时间复杂度为 $O(n+e)$ 。

结论：

稠密图适于在**邻接矩阵**上进行深度遍历；

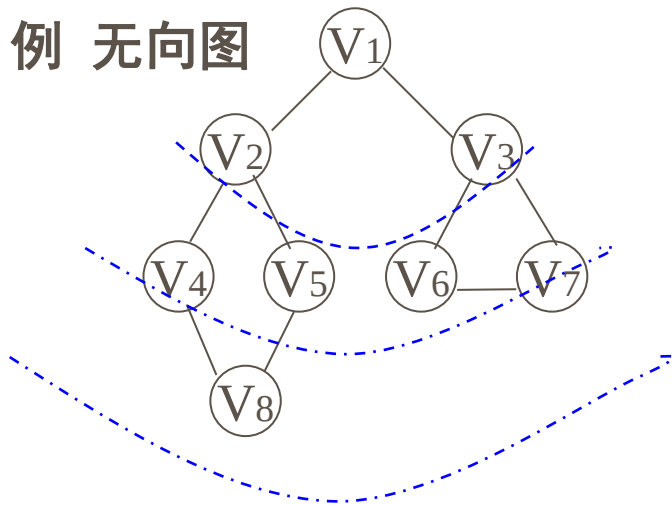
稀疏图适于在**邻接表**上进行深度遍历。

7.3.2 广度优先遍历

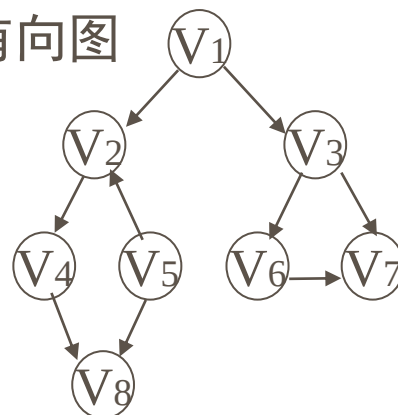
类似于树的按层次遍历

1. 从图中某顶点 v 出发，在访问 v 之后，
2. 依次访问 v 的各个未曾被访问过的邻接点，
3. 然后分别从这些邻接点出发依次访问它们的邻接点，并使“先被访问的顶点的邻接点”先于“后被访问的顶点的邻接点”被访问，直至图中所有已经被访问的顶点的邻接点都被访问到；
4. 若图中尚有顶点未曾被访问，则另选图中一个未曾被访问的顶点作起始点，访问该顶点，继续过程2、3，直至图中所有顶点都被访问到为止。

例 无向图



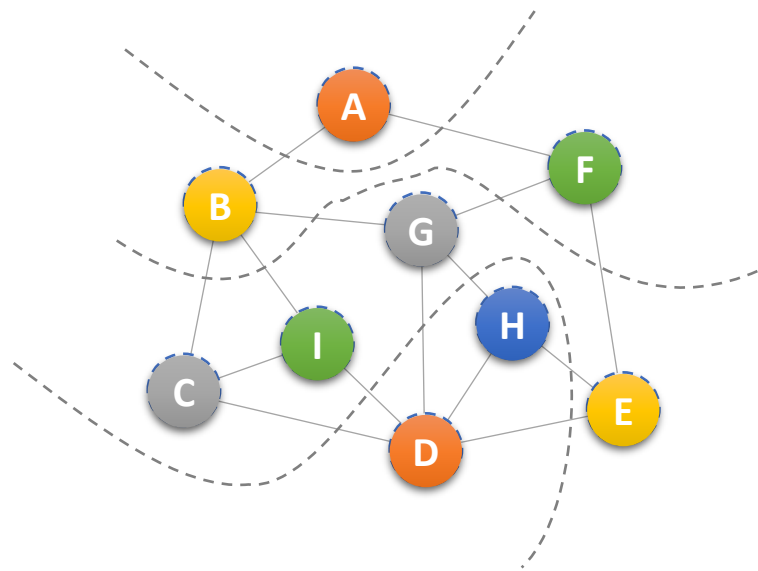
例 有向图



序列: **V1 V2 V3 V4 V5 V6 V7 V8** 序列: **V1 V2 V3 V4V6 V7 V8 V5**

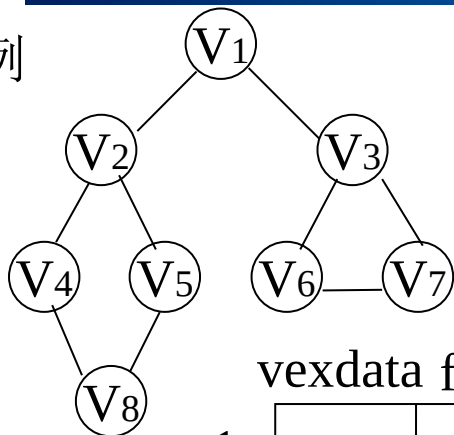
7.3.2 广度优先遍历

- 广度优先搜索是一种**分层**的搜索过程，每向前走一步可能访问一批顶点，不像深度优先搜索那样有回退的情况。
- 因此，广度优先搜索**不是一个递归的过程**，其算法也不是递归的。



7.3.2 广度优先遍历

例



思考题：按图的存储结构如何遍历？

思想同算法 $vi=v1$

广度遍历： $V1 \Rightarrow V3 \Rightarrow V2 \Rightarrow V7 \Rightarrow V6 \Rightarrow V5 \Rightarrow V4 \Rightarrow V8$

	vexdata	firstarc		adjvex	next
1	v1	→	3	→	2 ^
2	v2	→	5	→	4 → 1 ^
3	v3	→	7	→	6 → 1 ^
4	v4	→	8	→	2 ^
5	v5	→	8	→	2 ^
6	v6	→	7	→	3 ^
7	v7	→	6	→	3 ^
8	v8	→	5	→	4 ^

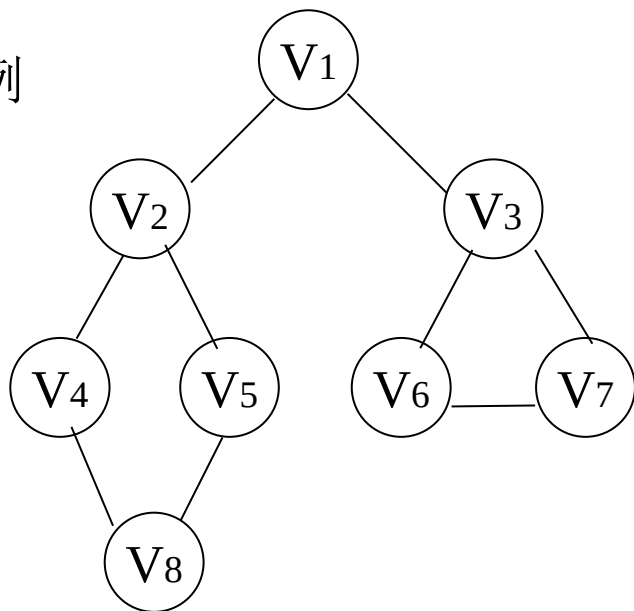
邻接矩阵如何？

7.3.2 广度优先遍历

广度优先搜索遍历图时，**对连通图，从起始点 V 到其余各顶点必定存在路径，并且访问顶点的次序是按顶点的路径长度递增进行的。**

例如：广度遍历序列： $V_1 V_2 V_3 V_4 V_5 V_6 V_7 V_8$

例



从 V_1 出发， $V_1 \rightarrow V_2$, $V_1 \rightarrow V_3$
的路径长度为1;

$V_1 \rightarrow V_4$, $V_1 \rightarrow V_5$, $V_1 \rightarrow V_6$, $V_1 \rightarrow V_7$
的路径长度为2;

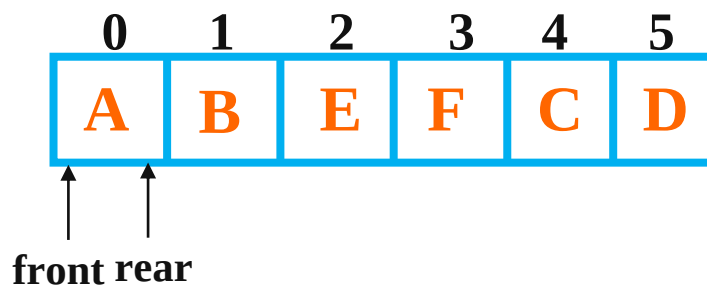
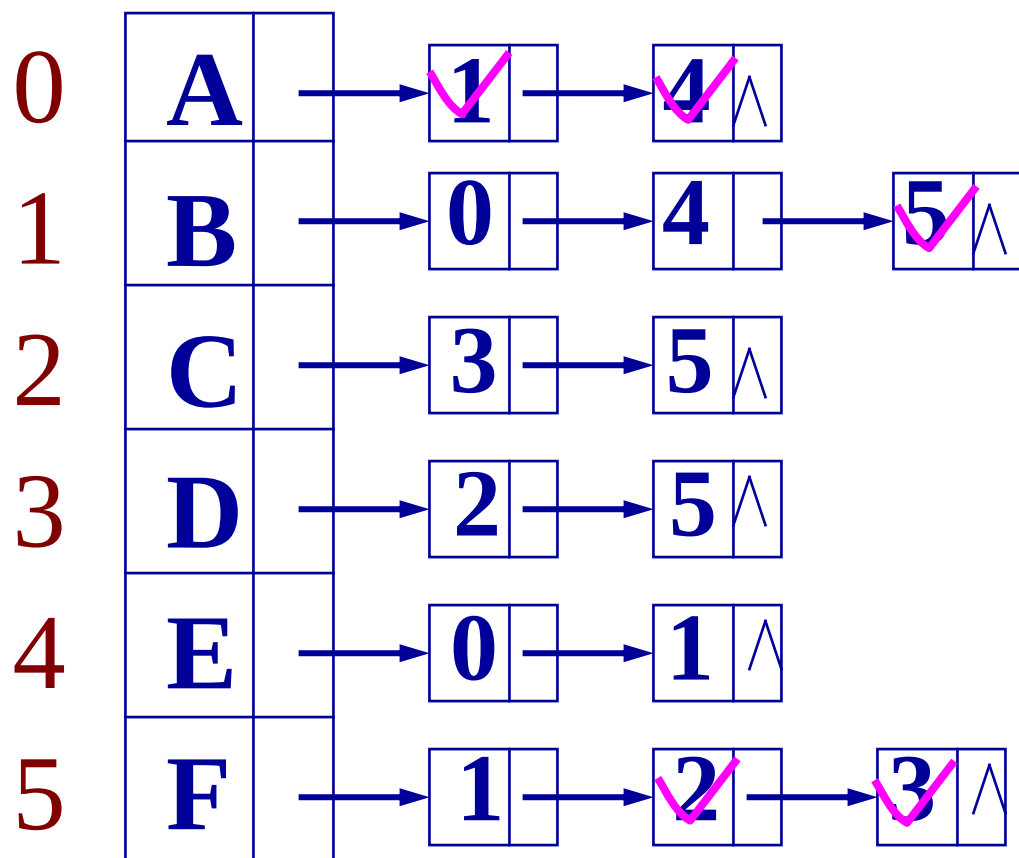
$V_1 \rightarrow V_8$ 的路径长度为3。

7.3.2 广度优先遍历

- ❑ 为避免重复访问, 需要一个辅助数组 **visited[n]**, 给被访问过的顶点加标记。
- ❑ 为了实现逐层（按顶点的路径长度递增）访问, 算法中需使用了一个队列, 以记忆正在访问的这一层和下一层的顶点, 以便于向下一层访问。

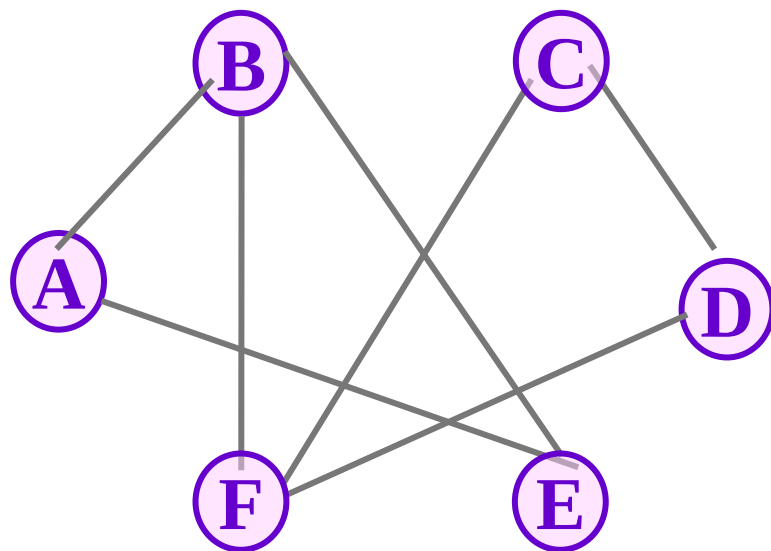
7.3.2 广度优先遍历

visited { ~~A~~, ~~B~~, ~~C~~, ~~D~~, ~~E~~, ~~F~~ }



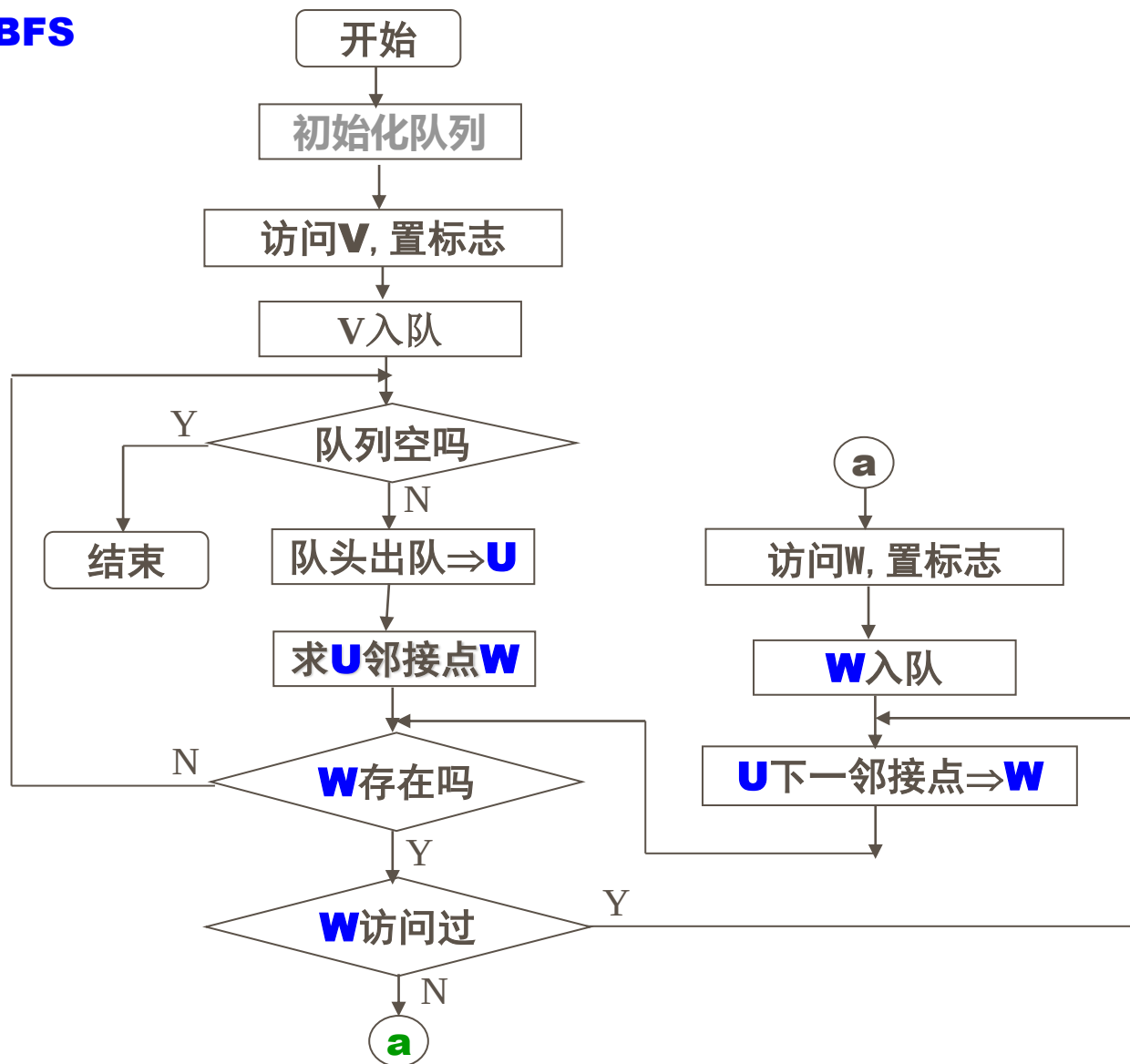
从A点开始遍历

A B E F C D



7.3.2 广度优先遍历

BFS



7.3.2 广度优先遍历

```
void BFSTraverse(Graph G, int v){ //对连通图G按广度优先非递归遍历
    for(v=0; v<G.vexnum; ++v)  visited[v] = FALSE;
    InitQueue(Q);                //置空的辅助队列Q
    for(v=0; v<G.vexnum; ++v)
        if(!visited[v]) { //v尚未访问
            visited[v] = TRUE ; Visit(v); // 访问v
            EnQueue(Q, v); // v入队列
            while (!QueueEmpty(Q)) {
                DeQueue(Q, u); // 队头元素出队并置为u
                for(w=FirstAdjVex(G, u); w!=0; w=NextAdjVex(G,u,w))
                    if ( ! visited[w]) {
                        visited[w]=TRUE; Visit(w);
                        EnQueue(Q, w); // 访问的顶点w入队列
                    } // if
            } // while
        }
    }
```

7.3.2 广度优先遍历

□ BFS算法效率分析

- ◆ 如果使用邻接矩阵，则BFS对于每一个被访问到的顶点，都要循环检测矩阵中的整整一行（ n 个元素），总的时间复杂度为 $O(n^2)$ 。
- ◆ 用邻接表来表示图，虽然有 $2e$ 个表结点，但只需扫描 e 个结点即可完成遍历，加上访问 n 个头结点的时间，时间复杂度为 $O(n+e)$ 。

7.3 图的遍历

例：求一条从顶点 i 到顶点 s 的简单路径

如：求从顶点 b 到顶点 k 的一条简单路径。

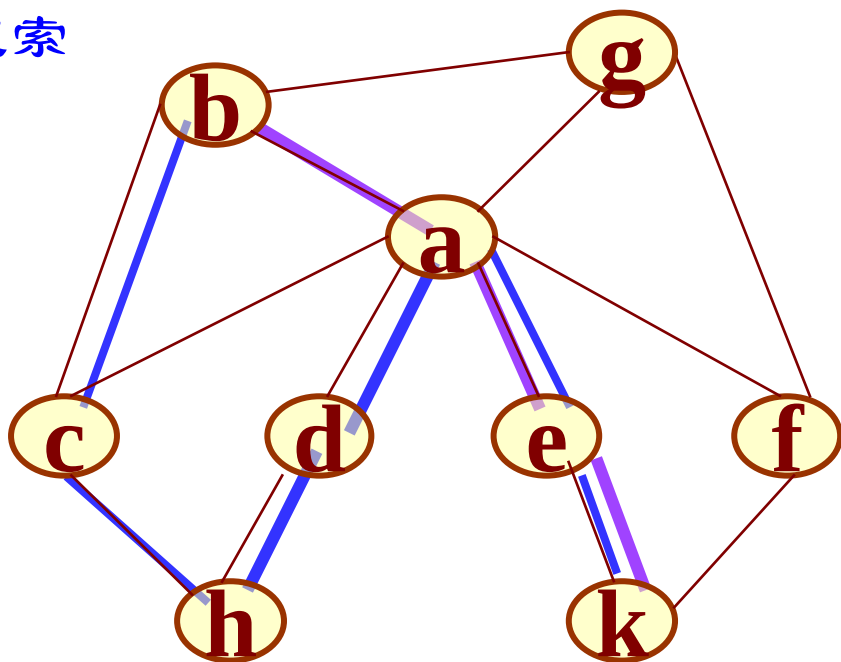
从顶点 b 出发进行深度优先搜索遍历。

假设找到的第一个邻接点是c，
则得到的结点访问序列为：

b c h d a e k f g,

假设找到的第一个邻接点是a，
则得到的结点访问序列为：

b a d h c e k f g。



思考：如何查找此路径？ 遍历过程中，检查是否到终点，是则止，否则继续。可能需进行若干次试探、回溯

7.3 图的遍历

结论:

1. 从顶点 i 到顶点 s , 若存在路径, 则从顶点 i 出发进行深度优先搜索, 必能搜索到顶点 s 。
2. 遍历过程中搜索到的顶点不一定是路径上的顶点, 则将该顶点从路径中删去, 如搜索路径为: $i, \dots, h, \underline{v'}, \dots$, 再从 h 出发重新探索下一条路径。
3. 由它出发进行的深度优先遍历整个图, 已经完成的顶点不是路径上的顶点, 则说明不存在路径 $(i, \dots, s)$ 。

7.3 图的遍历

```
void DFSearch( int v, int s, char *PATH)
```

```
{// 从第v个顶点出发递归地深度优先遍历图G，求得一条从v到s的简单路径，并记录在PATH中
```

```
    visited[v] = TRUE; // 访问第 v 个顶点 ， 能不被重复访问
```

```
    Append(PATH, getVertex(v)); // 第v个顶点加入路径 //k++;
```

```
    for (w=FirstAdjVex(G, v); w!=0&&!found;
```

```
        w=NextAdjVex(G, v, w) )
```

```
    { if (w==s) { found =TRUE; Append(PATH, w);exit(1) ;} //找到退出
```

```
        else if (!visited[w]) DFSearch(w, s, PATH);//加入w
```

```
    } //end for
```

```
    if (!found) Delete (PATH, v); // 从路径上删除顶点 v
```

```
} //
```

7.3 图的遍历

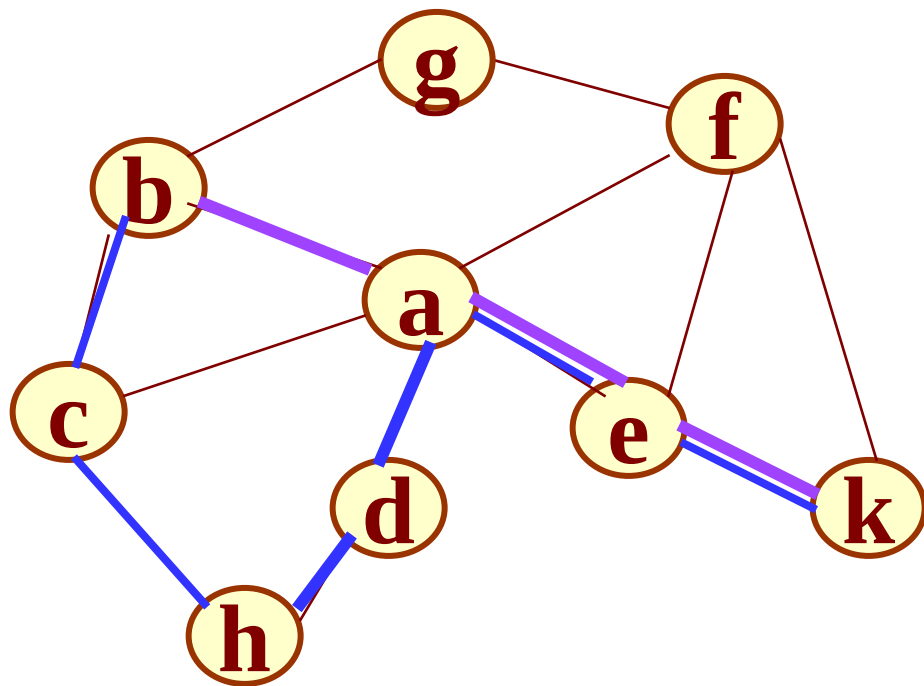
求有向图G的顶点v到s是否存在长度为k的简单路径

```
void DFSearch( int v, int s,int k, char *PATH) {  
//判断邻接表方式存储的有向图G的顶点v到s是否存在长度为k的简单路径,  
并记录在PATH中  
    if(v==s&& k==0)    //找到了一条路径,且长度符合要求  
        { found=TRUE; Append(PATH, v);return; }  
    else if(k>0) {  
        visited[v] = TRUE; // 访问第 v 个顶点  
        Append(PATH, getVertex(v)); L++; // 第v个顶点加入路径  
        for (w=FirstAdjVex(G, v); w!=0&&!found;  
                w=NextAdjVex(G, v, w) )  
            if (L==k&&w==s) { found = TRUE; Append(PATH, w); } //找到退出  
            else if (!visited[w]) DFSearch(w, s, k-1, PATH);  
        if (!found) Delete (PATH,v);    // 从路径上删除顶点 v  
    } //
```

7.3 图的遍历

求两个顶点之间的一条路径长度最短的路径

若两个顶点之间存在多条路径，则其中必有一条路径长度最短的路径。如何求得这条路径？



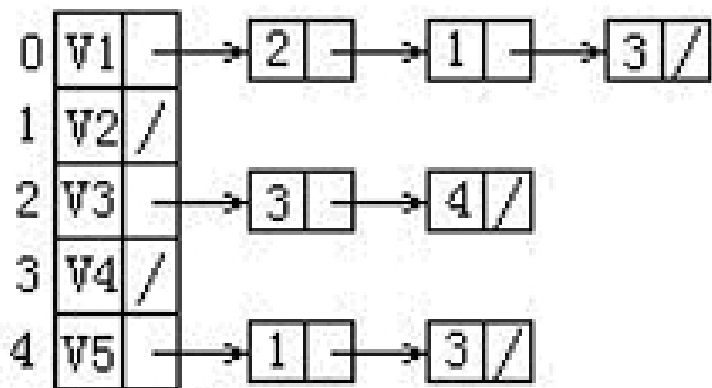
深度优先搜索访问顶点的**次序**取决于图的**存储结构**，而**广度优先**搜索访问顶点的次序是按“**路径长度**”渐增的次序。

因此，求路径长度最短的路径可以基于广度优先搜索遍历进行
(需修改链队列的结点结构及其入队列和出队列算法)

7.3 图的遍历

例，给定一有向图的邻接表如下。从顶点V1出发按深度优先搜索法进行遍历，则得到的一种顶点序列为：

- A. V1,V2,V3,V5,V4
- B. V1,V3,V4,V5,V2
- C. V1,V4,V3,V5,V2
- D. V1,V2,V4,V5,V3



7.3 图的遍历

例，已知一个图的邻接矩阵如下，则从顶点V1出发按深度优先搜索法进行遍历，可能得到的一种顶点序列为：

- A. V1,V2,V3,V4,V5,V6
- B. V1,V2,V4,V5,V6,V3
- C. V1,V3,V5,V2,V4,V6
- D. V1,V3,V5,V6,V4,V2

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

7.3 图的遍历

例：警察找到一个团伙的头目的一种方法是检查人们的电话。如果A和B之间有电话，我们说A和B是相关的。关系的权重被定义为两个人之间所有电话的总时间长度。“团伙”是由两个以上彼此相关的人组成的集群，总关系权重大于给定的K。在每个团伙中，总权重最大的是头。现在给定一个电话列表，找出团伙头目。

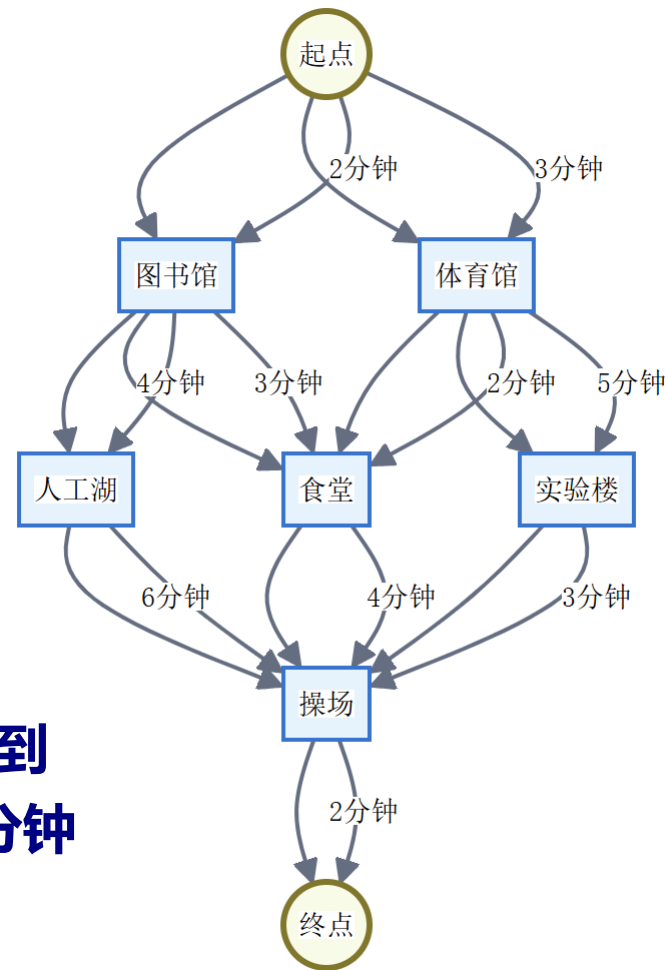
7.3 图的遍历

例：学校举办了一场定向越野比赛。参赛者需要从起点出发，经过校园内的多个检查点，最终到达终点。每个检查点都设有打卡装置，参赛者必须按要求完成打卡。

共6个检查点：图书馆(A)、体育馆(B)、人工湖(C)、食堂(D)、实验楼(E)、操场(F)
检查点之间的连线表示可行走的路径路径上标注的时间表示行走所需时间（分钟）

从起点出发，必须经过至少3个检查点，最后到达终点同一个检查点不能重复打卡需要在30分钟内完成全程。

请找出所有可能的路线方案在所有有效路线中，找出耗时最短的路线如果要求必须经过操场(F)，共有多少种不同的路线选择？



正在答疑
